

TRABAJO PRACTICO GRUPAL

- ESTRUCTURAS DE DATOS
- CONTRATO EQUALS/HASHCODE
- COMPLEJIDAD ALGORÍTMICA

Integrantes:

- Carabelli Alejo (51928)
- Ferrer Mara (51974)
- Petri Franco (52080)
- Ibañez Matias Daniel (52006)
- Medina Lautaro Miguel (52043)
- Garro Luka (51988)

Comisión
3k10

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Responder y Justificar:

1. Traza del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:

- **Al ejecutar `tarjetasValidas.add(t1)`:** Se agrega la tarjeta creada "t1" a la colección set tarjetasValidas con su código "ACC-101" y su nivel de acceso "2"; por lo tanto, se calcula el hashcode acorde a esos parámetros y se almacena en el bucket (cajón) correspondiente.

Traza:

- [Cajón 0] -> null
 - [Cajón 1] -> null
 - [Cajón 2] -> null
 - [Cajón 3] -> Referencia a Objeto {código: "ACC-101", nivelAcceso: 2}
 - [Cajón 4] -> null
-
- **Al ejecutar `t1.setNivelAcceso(5)`:** Se establece un nuevo nivel de acceso para la tarjeta "t1", pasando de valer 2 a valer 5. Sin embargo, el objeto permanece en el bucket asignado originalmente, no se reubica por más de tener un nuevo hash.
 - **Al ejecutar `tarjetasValidas.contains(t1)`:** Esta línea devuelve false, porque para verificar si la colección set tarjetasValidas contiene a la tarjeta t1, lo que hace es calcular su hashcode, va al bucket con ese hash, y ve que está vacío. Eso ocurre porque el hashcode ahora se calcula con un nivel de acceso de 5, y cuando se guardó el objeto en el bucket se hizo con un hashcode calculado con un nivel de acceso de 2.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

El objeto quedó guardado en el bucket original (por ejemplo, el "3"), que corresponde al hashcode calculado con el código "ACC-101" y al nivel de acceso "2". En cambio, al hacer `contains()`, Java lo va a buscar en el cajón "22", que corresponde al hashcode calculado con el código "ACC-101" y con el nivel de acceso "5". Por eso mismo lo encuentra vacío, porque el objeto no cambia físicamente de bucket.

Traza:

- [Cajón 0] -> null
- [Cajón 1] -> null
- [Cajón 2] -> null

- [Cajón 3] -> Referencia a Objeto {código: "ACC-101", nivelAcceso: 5}
- [Cajón 4] -> null

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Ocurre una fuga de memoria porque al objeto se lo coloca en su bucket original (por ejemplo, el 15), pero su hash cambia al modificarse el nivel de acceso, y ahora le corresponde otro bucket (por ejemplo, el 20) aunque físicamente no se lo mueva. Entonces, el Garbage Collector de Java no puede borrarlo porque físicamente sigue referenciado en un cajón, pero Java no puede encontrarlo con `contains()` ya que el hash es distinto. Por lo tanto, lógicamente "no existe" aunque físicamente sí esté.

Además, si se quisiera remover manualmente tampoco se podría, porque el método `remove()` también accede al objeto calculando su hash; en este caso, iría al bucket 20 y no al 15, que es donde realmente está. Si esto ocurre repetidamente, la memoria del servidor se llenaría de objetos a los cuales no se puede acceder ni eliminar.

Traza:

Búsqueda apunta al -> [Cajón 20] -> null (Retorna FALSE, no encuentra la referencia)

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

- Solución ajustando el criterio de la entidad: Si el código de la tarjeta es su clave primaria, esta será única para cada tarjeta. Por lo tanto, si el método `equals()` y el `hashCode()` se calculan solo en base al código y no al nivel de acceso, no importa cuántas veces se cambie el nivel de acceso, su hash no cambiará.

```

4      public class TarjetaAcceso {
5          private String codigo;
6          private int nivelAcceso;
7
8          public TarjetaAcceso(String codigo, int nivelAcceso) {
9              this.codigo = codigo;
10             this.nivelAcceso = nivelAcceso;
11         }
12
13         public void setNivelAcceso(int nivelAcceso) {
14             this.nivelAcceso = nivelAcceso;
15         }
16
17         @Override
18         public boolean equals(Object o) {
19             if (this == o) return true;
20             if (o == null || getClass() != o.getClass()) return false;
21             TarjetaAcceso tarjeta = (TarjetaAcceso) o;
22             return Objects.equals(codigo, tarjeta.codigo);
23         }
24
25         @Override
26         public int hashCode() {
27             return Objects.hash(codigo);
28         }
29     }

```

- Solución basada en inmutabilidad: Si se prefiere una inmutabilidad del objeto, se debe eliminar el método `setNivelAcceso()` de la clase para evitar cambios en esta, además de declarar la variable `nivelAcceso` como `private final` para que no se pueda cambiar su valor una vez se declara. (Esto significa que si a alguien le suben de nivel, se le debe sacar la tarjeta y darle una nueva con el nivel correspondiente). Alternativamente, se puede declarar la clase `TarjetaAcceso` como un record, lo cual la hace completamente inmutable sin necesidad de escribir código manualmente.

```
1 package org.example;
2 import java.util.*;
3
4 public class TarjetaAcceso {
5     private final String codigo;
6     private final int nivelAcceso;
7
8     public TarjetaAcceso(String codigo, int nivelAcceso) {
9         this.codigo = codigo;
10        this.nivelAcceso = nivelAcceso;
11    }
12
13    // Se elimina setNivelAcceso.
14    // equals() y hashCode() se mantienen igual.
15
16    @Override
17    public boolean equals(Object o) {
18        if (this == o) return true;
19        if (o == null || getClass() != o.getClass()) return false;
20        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
21        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo, tarjeta.codigo);
22    }
23
24    @Override
25    public int hashCode() {
26        return Objects.hash(codigo, nivelAcceso);
27    }
28 }
```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Responder y Justificar:

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación de la Elección
1. Auditoría	List	ArrayList	$O(n)$	$O(1)$	Como siempre se agregan al final, el ArrayList logra una complejidad de inserción de $O(1)$, ya que no debe desplazar los objetos como si se colocara en el medio. Conserva el orden de inserción y al requerir una búsqueda secuencial, su complejidad es de $O(n)$.
2. Ofertas	NavigableSet	TreeSet	$O(\log n)$	$O(\log n)$	Como no se necesita almacenar datos en formato Clave-Valor, se rechazan los duplicados y la colección debe quedar ordenada en base al precio, la implementación TreeSet es la correcta. Utiliza un árbol rojo-negro que mantiene los productos automáticamente ordenados, permitiendo extraer los extremos. El más barato a la izquierda y el más caro a la derecha del árbol.

3. Sesiones	Map	HashMap	O(1)	O(1)	Como cada usuario cuenta con un token identificador único y necesitamos que nos devuelva su sesión, usamos HashMap para tener la estructura de Clave-Valor. Asocia el token único del usuario (la clave) a su sesión; calcula el hash del token para acceder directamente a la posición en memoria y obtener el usuario a devolver.
-------------	-----	---------	------	------	---

Consigna 3: Optimización Algorítmica $O(n^2) \rightarrow O(n)$

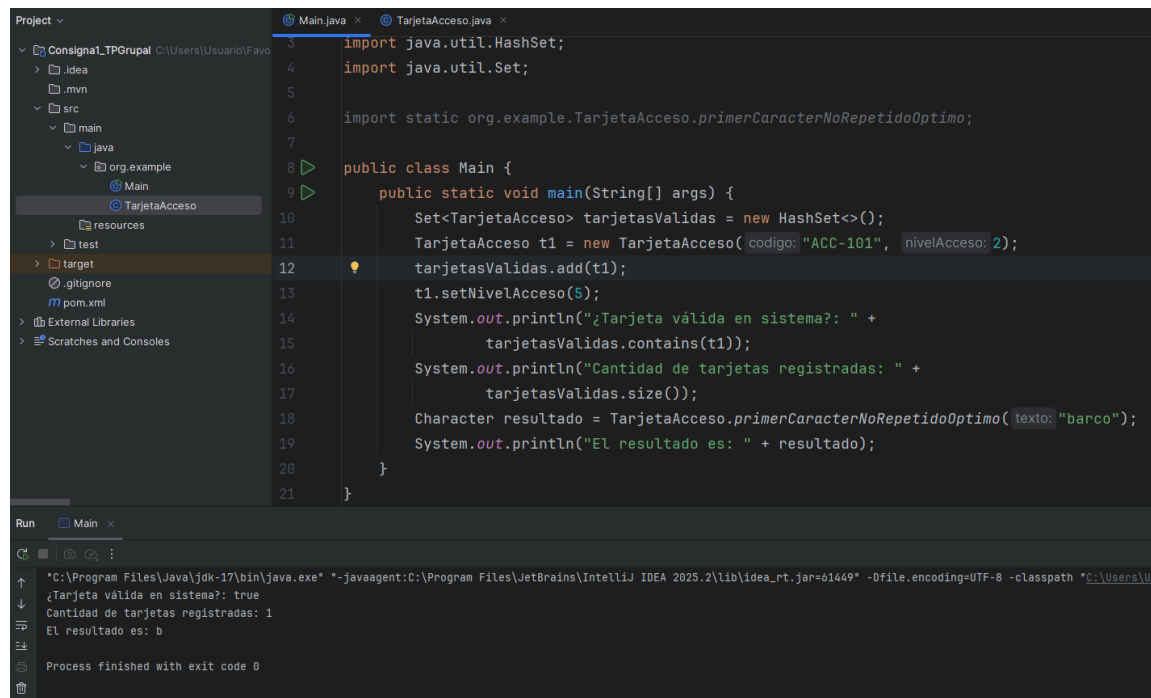
Responder y Justificar:

1. Refactorización a $O(n)$: Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

(Nota: Utilizamos LinkedHashMap para mantener el orden de inserción de los caracteres)

```
30 @ public static Character primerCaracterNoRepetidoOptimo(String texto) {
31     Map<Character, Integer> frecuencias = new LinkedHashMap<>();
32
33     for (int i = 0; i < texto.length(); i++) {
34         char c = texto.charAt(i);
35         frecuencias.put(c, frecuencias.getOrDefault(c, defaultValue: 0) + 1);
36     }
37
38     for (Map.Entry<Character, Integer> entry : frecuencias.entrySet()) {
39         if (entry.getValue() == 1) {
40             return entry.getKey();
41         }
42     }
43
44     return null;
45 }
46 }
```

Ejemplo de uso:



The screenshot shows an IDE with a project named 'Consigna1_TPGrupal'. The 'Main.java' file is open, showing the following code:

```
1 import java.util.HashSet;
2 import java.util.Set;
3
4 import static org.example.TarjetaAcceso.primerCaracterNoRepetidoOptimo;
5
6 public class Main {
7     public static void main(String[] args) {
8         Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();
9         TarjetaAcceso t1 = new TarjetaAcceso( codigo: "ACC-101", nivelAcceso: 2);
10         tarjetasValidas.add(t1);
11         t1.setNivelAcceso(5);
12         System.out.println("¿Tarjeta válida en sistema?: " +
13             tarjetasValidas.contains(t1));
14         System.out.println("Cantidad de tarjetas registradas: " +
15             tarjetasValidas.size());
16         Character resultado = TarjetaAcceso.primerCaracterNoRepetidoOptimo( texto: "barco");
17         System.out.println("El resultado es: " + resultado);
18     }
19 }
```

The 'Run' console shows the output of the program:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2\lib\idea_rt.jar=61449" -Dfile.encoding=UTF-8 -classpath "C:\Users\Us...
¿Tarjeta válida en sistema?: true
Cantidad de tarjetas registradas: 1
El resultado es: b
Process finished with exit code 0
```

2. Análisis de Colisiones: Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

- Si se utiliza un HashMap y dos objetos generan el mismo hashCode(), ambos intentan ser ubicados y caen en el mismo bucket (cajón). A esto se le llama Colisión de Hash. El segundo objeto no sobrescribe al primero, sino que se enlaza a él mediante una técnica llamada Encadenamiento. Se forma una LinkedList o lista enlazada donde ambos objetos quedan guardados como nodos conectados, uno detrás del otro, en el mismo espacio o bucket.
- Para no perder información, la estructura calcula el hashCode(), accede al bucket, y al ver que hay más de un elemento, recorre secuencialmente los objetos que están enlazados dentro de este espacio usando obligatoriamente el método equals() sobre la clave para identificar de manera precisa cuál de los objetos es el correcto, descartando la simple coincidencia numérica.

El problema de esto es que encontrar el espacio correcto sí tiene una complejidad ideal de $O(1)$, pero luego buscar el objeto correcto recorriendo secuencialmente la lista enlazada que se forma dentro del espacio degrada la complejidad a $O(n)$.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si s2 debe ingresarse o desecharse.

Si guardamos ambas cadenas en un `HashSet`, la colección va a tener 2 elementos. Sabemos que el hashcode de ambas instancias es el mismo, por lo tanto cuando java quiera asignar la segunda instancia al espacio en el que ya se encuentra la primera lo que hará es ejecutar el método `equals()` para corroborar si el elemento es distinto o no. El método `equals` correspondiente a la clase `String` lo que hace es comparar el texto dentro del objeto. Como FB es distinto a Ea, el método dirá que no es un duplicado (devuelve false) y permite el ingreso de esta segunda instancia a ese mismo bucket enlazando al primero (formando una linked list)

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

En el peor de los casos, donde todos los objetos obtengan el mismo hashcode, todos caerán al mismo bucket y la complejidad aumentará a $O(n)$. Aunque en java 8 en adelante, cuando hay más de 8 objetos en un mismo espacio se forma un árbol rojo-negro similar a un `TreeMap` reduciendo la complejidad a $O(\log n)$